Problem Models:

To start coding this out, the main hurdle was getting a solid grip on the problem models first. I ran everything in Python using a Jupyter Notebook. To represent the 4 x 4 board, I went with a 16-element tuple. Following the prompt's setup, raw input strings use _ for the empty spot and / to split up rows. An input like this: 1 2 7 3 / 5 6 _ 4 / 9 10 11 8 / 13 14 15 12, gets converted straight into a tuple: (1, 2, 7, 3, 5, 6, 0, 4, 9, 10, 11, 8, 13, 14, 15, 12), where 0 acts as the blank tile. Tuples were the obvious choice here because they're immutable and hashable in Python, which makes set lookups fast and easy when managing state history.

Valid moves are shifting tiles Right, Left, Up, or Down (R, L, U, D), assuming the swap stays within the 4 x 4 grid boundaries. Every move triggers a goal test to see if we reached the solved board: numbers 1 through 15 in order from left to right, top to bottom, with 0 sitting in the bottom-right corner. Every tile swap costs 1 unit.

I relied heavily on tuples for the railroad problem to hold coordinates (latitude, longitude) for geographic distance math. I also use tuples in my priority queue definitions to help pair information, or specifically to pair the heuristic distance and the city name. This kept the graph representation clean and easy to iterate over.

An action here is just moving along an existing rail line from one station to a connected neighbor. The goal test simply checks if the current station matches our target (like San Francisco).

Strategies used included:

- DFS: Frontier was managed as a Last-In, First-out stack, meaning that the most recently added element is the one that is popped. This mimics DFS because it fully explores a

certain branch of a tree until there are no more neighbors before exploring the next branch.

- BFS: This was used as the base for my DFS model, with the simple change that instead of popping the most recent element, the element popped is actually the element that had been in it the longest, making it explore full levels of the tree before going deeper into it.
- UCS: Frontier was managed as a prior queue ordered by cumulative post cost in this version, which helped guarantee optimal post cost on weighted graphs.
- Greedy BEst-First Search: Priority queue that is ordered strictly by the heuristic estimate at each decision it has to make. This prioritizes speed over being fully optimized. Basically, it will always choose the shortest path available.
- A* Search: Priority queue that is ordered by the total estimated cost function ($f = g + h$). This guarantees that the optimal path is taken, and takes into account the distance traveled to that point as well as the heuristic function so that it can take the shortest path moving forward as well. It adds these two values, and the priority queue is based on their sum.

Both problems also used a closed set in order to make sure that duplicates are not explored. This helps avoid getting stuck in a loop. For DFS and BFS, once a node is popped from the frontier, it is then also added to the closed set and ignored, then, if it is generated again. For UCS and A*, a new path is only reopened if it is shorter than the existing path, which is explicitly checked in the code.

For the 15-Puzzle, whether it can be solved or not is verified before the execution of any of the code in order to avoid wasting resources on unsolvable boards. Whether it can be solved or

not is determined by comparing permutation parity and the Manhattan distance of the blank space to the goal index, which is also explicitly in the code.

Heuristics:

- 15-Puzzle: Uses Manhattan distance, which sums up the horizontal and vertical grid steps each numbered tile needs to reach its target slot (ignoring the blank space entirely). For example, given 1 2 7 3 / 5 6 _ 4 / 9 10 11 8 / 13 14 15 12, the distance works out to 5. Tiles 3, 4, 7, 8, and 12 are out of place, but each is only 1 step away from where it belongs.
- Railroad: Uses straight-line great-circle distance between the current station's coordinates and the goal station's coordinates.

Results:

Railroad table:

Strategy	found?	cost/ miles	stops	expanded	generated	Peak frontier	Runtime (ms)	optimal?	note
DFS	yes	3551	14	109	110	16	0.51	no	+352
BFS	yes	7907	41	77	199	126	0.40	no	+4708
UCS	yes	3199	17	104	127	14	0.65	yes	
Greedy	yes	3253	16	17	50	34	0.39	no	+54

A*	yes	3199	17	89	127	28	0.82	yes	

15-Puzzle table:

For the following board:

1 2 7 3 / 5 6 _ 4 / 9 10 11 8 / 13 14 15 12

Strategy	found?	cost/ moves	stops	expanded	generated	Peak frontier	Runtime (ms)	optimal?	note
Greedy	yes	5	5	5	12	7	0.17	yes	
A*	yes	5	5	5	12	7	0.25	yes	

_ 2 4 8 / 1 7 3 6 / 10 5 11 12 / 9 14 13 15

Strategy	found?	cost/ moves	stops	expanded	generated	Peak frontier	Runtime (ms)	optimal?	note
Greedy	yes	76	76	378	822	442	0.0099	no	
A*	yes	28	28	15,671	30,434	14,764	0.49	yes	

Analysis:

1. BFS does not take into account the distance of an edge or of travel. This search method simply moves its way down through each level of the tree iteratively, using a First-In, First-Out queue. This differs from A*, which is intelligent in that it tracks the amount of distance covered to the current city and sums it with the heuristic distance to the lowest possible neighbor. This intelligent approach is why A* is more optimal and returns a different route than BFS.
2. Greedy search ignores the distance covered to get to the current city, as this search mechanism strictly focuses on the shortest possible path from that current city to one of its neighbors, while A* sums both of those values in order to determine the optimal solution. In the railroad problem, this makes greedy search have a lower cost, fewer expanded nodes, and a quicker runtime. However, it also sacrifices optimization, as A* will be the truly most optimal solution.
3. The fewest railroad nodes expanded by Greedy search. However, it did not have the cheapest cost, which belongs to both UCS and A*. This is because greedy search sacrifices optimization for speed, which shows that search time does not equate directly to search quality and optimization.
4. A* had a shorter solution length in terms of moves, while greedy expanded fewer nodes and had a smaller peak frontier. The heuristic used was the Manhattan distance, which guided greedy to a goal but not an optimal one, because while greedy was strictly using this heuristic, it was inadvertently making its own job harder. When this search algorithm is moving tiles to their targets, it inadvertently moves other tiles away from their goal positions, making it a longer overall sequence.

5. A* is optimal under the assumption that the heuristic is admissible, which means that it never overestimates the true remaining cost to the goal. If the heuristic was inadmissible, that would make it possible for A* to overestimate the optimal path and skip over it in favor of a less optimal path.
6. If the memory was the limit, the best option would be to measure the peak frontier, or the total nodes stored in memory/the closed set). For this one, greedy search would be the best option. If solution quality was the limit, the path cost would be the most important option to track. A* would be th best option because it strictly guarantees the optimal solution.
7. N/a
8. N/a

Reproducibility

- Environment: Python 3.10.12 in Jupyter Notebook on Windows.
- Imports Used: heapq, json, math.
- Use Github README.md, which brings access to the code and files necessary to run the program on your own. Code is editable simply. For 15-puzzle, the list of strings that contains the strings of example puzzles can be edited to reproduce the result for different starting points. For the railroad problem, a different destination can be chosen for the program and a different starting point can be chosen.
- Each search mechanism is labeled and can be repurposed (this is actually what I did between the railroad and puzzle problems).